



UNIVERSITÀ DEGLI STUDI DI SALERNO



MealPlannerApp

Relazione tecnica di progettazione e sviluppo

Progetto universitario

Corso: Mobile Programming

Anno accademico: 2025/2026

Traccia: Meal Planner & Smart Pantry

Gruppo 18

Componente	Matricola
Salvatore Moccia	0612708799
Davide Martelli	0612709368
Simone Iannone	0612708913
Lorenzo Nevola	0612709366

Indice

1	Introduzione	1
1.1	Obiettivo dell'applicazione	1
1.2	Utenti target e problema affrontato	1
1.3	Scenario operativo generale	1
2	Analisi dei requisiti	2
2.1	Requisiti funzionali	2
2.2	Requisiti non funzionali	3
2.3	Casi d'uso	3
2.3.1	UC-01 — Gestione ricette	4
2.3.2	UC-02 — Gestione dispensa	4
2.3.3	UC-03 — Pianificazione pasti	5
2.3.4	UC-04 — Generazione lista della spesa	5
2.3.5	UC-05 — Acquisto e aggiornamento dispensa	6
2.3.6	UC-06 — Consultazione dashboard	6
2.4	Copertura della traccia	6
2.5	Feature avanzate	7
2.6	Funzionalità escluse e limitazioni	7
3	Progettazione dell'app	8
3.1	Struttura generale	8
3.2	Navigazione	8
3.3	Modello dati	9
3.4	Flusso dei dati	9
4	Interfaccia e schermate	9
4.1	Dashboard	10
4.2	Gestione delle ricette	10
4.3	Gestione della dispensa	11
4.4	Pianificazione dei pasti	12
4.5	Lista della spesa	13
5	Scelte tecnologiche	14
5.1	Framework e versione Expo	14
5.2	Librerie principali	14
5.3	Gestione della persistenza	15
6	Implementazione	15
6.1	Organizzazione del codice	15
6.2	Gestione dello stato	15
6.3	CRUD ricette	16
6.4	CRUD dispensa	16
6.5	Pianificazione dei pasti	16
6.6	Lista della spesa automatica	16
6.7	Dashboard e statistiche	16
7	Conclusioni	17

1 Introduzione

1.1 Obiettivo dell'applicazione

MealPlannerApp è un'applicazione mobile, sviluppata in React Native con Expo. Copre quattro aree: ricette, dispensa, pianificazione dei pasti e lista della spesa. Una dashboard raccoglie i dati delle quattro aree e all'avvio ne mostra una sintesi.

Il punto di partenza è che le schermate non lavorino come applicazioni separate messe nella stessa app. Una ricetta entra in un pasto pianificato; gli ingredienti di quel pasto vengono confrontati con le scorte in dispensa; ciò che manca diventa una voce nella lista della spesa. Quando l'utente segna i prodotti come acquistati, questi rientrano in dispensa e il ciclo si chiude.

1.2 Utenti target e problema affrontato

L'applicazione è pensata per chi cucina con una certa regolarità e vuole tenere in ordine ricette, scorte e acquisti: studenti, lavoratori, famiglie, ma anche chi vive da solo. Le esigenze ricorrenti sono:

- consultare rapidamente le ricette salvate;
- sapere quali ingredienti ci sono già in casa;
- pianificare i pasti dei giorni successivi;
- costruire una lista della spesa basata sui fabbisogni reali;
- ridurre sprechi, dimenticanze e acquisti inutili.

Il problema concreto è la dispersione delle informazioni. Le ricette stanno da una parte, quello che c'è in dispensa nella memoria di chi cucina, la lista della spesa su un foglietto a parte: dati che non comunicano e che portano a doppioni e dimenticanze. MealPlannerApp li raccoglie in un unico posto e riutilizza ogni informazione dove serve, senza chiedere all'utente di reinserirla.

1.3 Scenario operativo generale

In fase di progettazione abbiamo preso come riferimento un flusso d'uso concreto:

1. l'utente consulta le ricette salvate e ne inserisce una nuova con ingredienti, categoria, tempo di preparazione e difficoltà;
2. registra un prodotto in dispensa indicando quantità, unità di misura e data di scadenza;
3. pianifica un pasto scegliendo una ricetta e assegnandola a uno dei prossimi sette giorni;
4. l'app genera la lista della spesa confrontando gli ingredienti dei pasti pianificati con quello che c'è in dispensa;
5. dopo l'acquisto, l'utente segna gli elementi come acquistati e li sposta in dispensa;
6. infine apre la dashboard per controllare riepiloghi, prodotti in scadenza e ingredienti mancanti.

2 Analisi dei requisiti

2.1 Requisiti funzionali

I requisiti funzionali elencano cosa può fare l'utente e i comportamenti che reggono il ciclo operativo: gestire le ricette, aggiornare la dispensa, pianificare i pasti e generare la lista della spesa.

ID	Descrizione	Categoria	Priorità	Stato
RF-01	Visualizzare l'elenco delle ricette salvate, con informazioni sintetiche su categoria, difficoltà e tempo di preparazione.	Ricette	Alta	Implementato
RF-02	Inserire, modificare ed eliminare una ricetta, includendo descrizione, ingredienti, porzioni, note e immagine opzionale tramite URL.	Ricette	Alta	Implementato
RF-03	Consultare il dettaglio di una ricetta e accedere alle operazioni di modifica o cancellazione.	Ricette	Alta	Implementato
RF-04	Ricerca le ricette per nome o categoria e filtrare quelle preparabili con gli ingredienti disponibili.	Ricette	Media	Implementato
RF-05	Visualizzare, inserire, modificare ed eliminare prodotti presenti in dispensa, includendo quantità, unità di misura e scadenza.	Dispensa	Alta	Implementato
RF-06	Evidenziare i prodotti scaduti o prossimi alla scadenza e renderli consultabili tramite filtro dedicato.	Dispensa	Media	Implementato
RF-07	Pianificare i pasti dei prossimi sette giorni, associando data, tipo di pasto e ricetta.	Meal Plan	Alta	Implementato
RF-08	Modificare o rimuovere un pasto pianificato e consultare il dettaglio della ricetta associata.	Meal Plan	Media	Implementato
RF-09	Generare automaticamente la lista della spesa confrontando gli ingredienti pianificati con i prodotti disponibili.	Spesa	Alta	Implementato
RF-10	Gestire manualmente gli elementi della lista della spesa, con modifica in linea, eliminazione e marcatura come acquistati.	Spesa	Alta	Implementato

ID	Descrizione	Categoria	Priorità	Stato
RF-11	Trasferire gli elementi acquistati dalla lista della spesa alla dispensa, aggiornando quantità esistenti o creando nuovi prodotti.	Spesa	Media	Implementato
RF-12	Mostrare una dashboard con statistiche, riepiloghi settimanali, prodotti in scadenza e ingredienti mancanti.	Dashboard	Media	Implementato

2.2 Requisiti non funzionali

I requisiti non funzionali riguardano le qualità del sistema e i vincoli di esecuzione. Le priorità sono state tre: chiarezza dell'interfaccia, persistenza locale dei dati e compatibilità con l'ambiente Expo.

ID	Descrizione	Categoria	Stato
RNF-01	I dati devono restare disponibili anche dopo la chiusura e la riapertura dell'app.	Persistenza	Implementato
RNF-02	La navigazione deve essere divisa in sezioni riconoscibili, coerenti con i domini funzionali.	Usabilità	Implementato
RNF-03	Le schermate devono essere pensate per orientamento portrait e uso da dispositivo mobile.	Interfaccia	Implementato
RNF-04	Il codice deve usare TypeScript per rendere esplicite le strutture dati principali.	Manutenibilità	Implementato
RNF-05	La soluzione deve funzionare senza backend remoto, su dati locali e mock iniziali.	Architettura	Implementato
RNF-06	Le funzionalità principali devono essere accessibili senza configurazioni esterne complesse.	Eseguibilità	Implementato

2.3 Casi d'uso

L'attore è sempre l'utente finale. I casi d'uso descrivono le interazioni principali e, soprattutto, i punti in cui ricette, dispensa, pianificazione, lista della spesa e dashboard si scambiano dati.

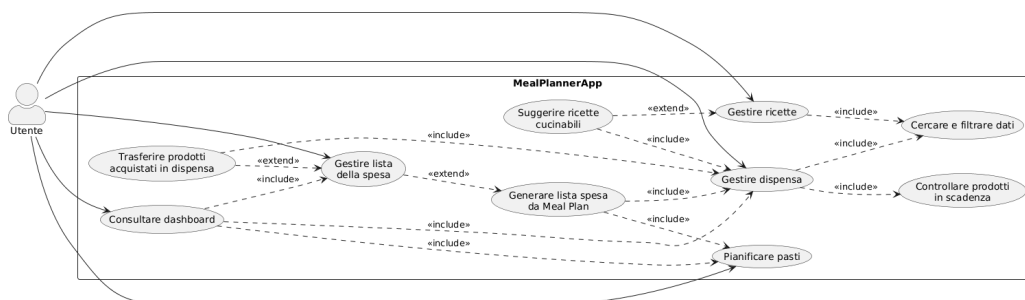


Figura 1: Diagramma generale dei casi d'uso principali

2.3.1 UC-01 — Gestione ricette

- **Attore primario:** Utente.
- **Precondizioni:** L'app è avviata e lo store delle ricette è disponibile.
- **Trigger:** L'utente apre la sezione Ricette.
- **Scenario principale:**
 1. Il sistema mostra l'elenco delle ricette salvate.
 2. L'utente cerca o filtra le ricette per nome, categoria o disponibilità degli ingredienti.
 3. L'utente apre il dettaglio di una ricetta.
 4. L'utente inserisce una nuova ricetta oppure modifica o elimina una esistente.
 5. Il sistema aggiorna lo store locale e rende i dati disponibili alle altre sezioni.
- **Scenari alternativi:**
 - Nessuna ricetta presente: il sistema mostra uno stato vuoto e invita ad aggiungerne una.
 - Campi obbligatori mancanti: il salvataggio viene impedito e i dati non validi restano fuori dallo store.
- **Postcondizione:** L'archivio delle ricette è consultabile e aggiornato.
- **Requisiti collegati:** RF-01, RF-02, RF-03, RF-04.

2.3.2 UC-02 — Gestione dispensa

- **Attore primario:** Utente.
- **Precondizioni:** L'app è avviata e lo store della dispensa è disponibile.
- **Trigger:** L'utente apre la sezione Dispensa.
- **Scenario principale:**
 1. Il sistema mostra i prodotti in dispensa.
 2. L'utente cerca i prodotti per nome o categoria.
 3. L'utente aggiunge un prodotto con quantità, unità di misura, categoria e data di scadenza.
 4. L'utente modifica o rimuove i prodotti esistenti.
 5. Il sistema segnala i prodotti scaduti o vicini alla scadenza.
- **Scenari alternativi:**
 - Data di scadenza assente: il sistema stima una scadenza in base alla categoria.
 - Nessun prodotto corrispondente alla ricerca: il sistema mostra una lista vuota.
- **Postcondizione:** La dispensa contiene dati aggiornati, usabili da dashboard, meal plan e lista della spesa.
- **Requisiti collegati:** RF-05, RF-06.

2.3.3 UC-03 — Pianificazione pasti

- **Attore primario:** Utente.
- **Precondizioni:** Sono presenti ricette salvate.
- **Trigger:** L'utente apre la sezione Meal Plan.
- **Scenario principale:**
 1. Il sistema mostra i prossimi sette giorni.
 2. L'utente sceglie una data, un tipo di pasto e una ricetta.
 3. Il sistema salva il pasto pianificato nello store locale.
 4. L'utente modifica o rimuove il pasto pianificato.
 5. L'utente apre il dettaglio della ricetta associata.
- **Scenari alternativi:**
 - Nessuna ricetta disponibile: il sistema non consente di completare la pianificazione.
 - Giorno senza pasti: il sistema mostra uno stato vuoto per quel giorno.
- **Postcondizione:** Il piano settimanale è aggiornato.
- **Requisiti collegati:** RF-07, RF-08.

2.3.4 UC-04 — Generazione lista della spesa

- **Attore primario:** Utente.
- **Precondizioni:** Sono presenti pasti pianificati e prodotti in dispensa.
- **Trigger:** L'utente avvia la generazione automatica nella sezione Spesa.
- **Scenario principale:**
 1. Il sistema recupera i pasti pianificati.
 2. Il sistema legge gli ingredienti delle ricette associate.
 3. Il sistema confronta gli ingredienti richiesti con i prodotti in dispensa.
 4. Il sistema calcola le quantità mancanti.
 5. Il sistema genera gli elementi automatici della lista della spesa.
- **Scenari alternativi:**
 - Nessun pasto pianificato: il sistema non genera nuovi elementi automatici.
 - Ingrediente già disponibile in quantità sufficiente: non viene aggiunto alla lista.
 - Unità di misura diverse: il sistema non effettua conversioni.
- **Postcondizione:** La lista della spesa contiene gli ingredienti mancanti rispetto al meal plan.
- **Requisiti collegati:** RF-09, RF-10.

2.3.5 UC-05 — Acquisto e aggiornamento dispensa

- **Attore primario:** Utente.
- **Precondizioni:** La lista della spesa contiene almeno un elemento.
- **Trigger:** L'utente segna uno o più elementi come acquistati e ne chiede lo spostamento in dispensa.
- **Scenario principale:**
 1. L'utente segna un elemento della lista come acquistato.
 2. Il sistema aggiorna lo stato dell'elemento.
 3. L'utente sposta gli acquistati in dispensa.
 4. Il sistema controlla se il prodotto c'è già con stesso nome e stessa unità di misura.
 5. Se il prodotto esiste, il sistema somma la quantità acquistata.
 6. Se il prodotto non esiste, il sistema lo crea in dispensa.
- **Scenari alternativi:**
 - Nessun elemento acquistato: la dispensa resta invariata.
 - Prodotto con unità diversa: il sistema crea o mantiene elementi separati.
- **Postcondizione:** La dispensa è aggiornata con i prodotti acquistati.
- **Requisiti collegati:** RF-10, RF-11.

2.3.6 UC-06 — Consultazione dashboard

- **Attore primario:** Utente.
- **Precondizioni:** L'app contiene dati su ricette, dispensa, meal plan o lista della spesa.
- **Trigger:** L'utente apre la schermata Dashboard.
- **Scenario principale:**
 1. Il sistema legge i dati dagli store locali.
 2. Il sistema calcola statistiche e riepiloghi.
 3. Il sistema mostra numero di ricette, pasti pianificati, prodotti in scadenza e ingredienti mancanti.
 4. L'utente consulta le informazioni operative principali.
- **Scenari alternativi:**
 - Dati assenti o insufficienti: il sistema mostra valori a zero o sezioni vuote.
- **Postcondizione:** L'utente ha una visione sintetica dello stato dell'app.
- **Requisiti collegati:** RF-12.

2.4 Copertura della traccia

La tabella riepiloga come il progetto risponde ai punti della traccia. La verifica è tenuta separata dall'elenco dei requisiti per non confondere l'analisi del prodotto con l'aderenza alla consegna.

Richiesta della traccia	Implementazione nel progetto	Stato
CRUD sui dati principali	Inserimento, visualizzazione, modifica ed eliminazione su ricette, dispensa, pasti pianificati e lista della spesa.	Coperto
Navigazione multi-schermata	Bottom tab navigator con stack interni per le sezioni con dettaglio e form.	Coperto
Persistenza locale	Store Zustand persistenti su AsyncStorage, con una chiave per dominio.	Coperto
Ricerca o filtri	Ricerca ricette, ricerca dispensa, filtro prodotti in scadenza e filtro ricette preparabili.	Coperto
Riepiloghi/statistiche	Dashboard con statistiche su ricette, pasti, scadenze e ingredienti mancanti.	Coperto
Feature avanzate	Lista della spesa automatica, ricette preparabili, gestione scadenze e trasferimento acquistati.	Coperto
Backend remoto	Non richiesto dalla traccia; il progetto lavora in locale, senza servizi remoti.	Non previsto

2.5 Feature avanzate

Il progetto va oltre le due funzionalità avanzate richieste. Tutte sono agganciate ai flussi principali e servono a tenere collegati meal plan, dispensa e lista della spesa:

- **Generazione automatica della lista della spesa:** dai pasti pianificati l'app raccoglie gli ingredienti, toglie quello che c'è già in dispensa e crea le voci marcate come automatiche.
- **Suggerimento di ricette preparabili:** il pulsante “Cosa posso cucinare oggi?” nella sezione Ricette mostra solo le ricette i cui ingredienti bastano con le scorte attuali.
- **Gestione delle scadenze:** la dispensa segnala i prodotti scaduti o in scadenza entro sette giorni, e la dashboard ne tiene il conto.
- **Trasferimento degli acquistati in dispensa:** gli elementi segnati come acquistati aggiornano le quantità esistenti o creano nuovi prodotti.

2.6 Funzionalità escluse e limitazioni

Alcune funzioni avrebbero senso nel dominio, ma sono rimaste fuori per tenere il progetto entro la traccia e proporzionato a un contesto didattico:

- autenticazione utente e sincronizzazione cloud;
- condivisione delle ricette tra utenti;
- notifiche push per i prodotti in scadenza;
- conversione automatica tra unità di misura diverse, ad esempio grammi e chilogrammi;
- calendario mensile avanzato o pianificazione oltre la settimana;
- scansione del codice a barre o importazione automatica dei prodotti.

Non c'è backend e non vengono fatte chiamate a servizi remoti. Le scadenze si inseriscono a mano come stringhe in formato YYYY-MM-DD; il confronto tra ingredienti e dispensa funziona solo a parità di nome e unità di misura; la validazione dei form copre i casi principali; le immagini delle ricette arrivano da URL e non vengono salvate in locale.

3 Progettazione dell'app

3.1 Struttura generale

Il codice è diviso in moduli per tipo di responsabilità: schermate, navigatori, store, tipi e utilità. È una suddivisione che si legge bene e tiene separate le parti. Tutto sta sotto `src/`:

- `screens/`: le schermate, raggruppate per dominio;
- `navigation/`: navigazione principale e stack interni;
- `store/`: gli store Zustand persistenti;
- `types/`: le interfacce TypeScript del dominio;
- `utils/`: tema grafico e dati mock iniziali.

Il punto di ingresso è `App.tsx`: carica il supporto ai valori random che serve a `uuid`, configura la `StatusBar` e monta il navigatore principale. `index.js` registra il componente root tramite Expo.

3.2 Navigazione

La navigazione principale usa `@react-navigation/bottom-tabs`. La tab bar è il livello globale e dà accesso a cinque sezioni: Dashboard, Ricette, Dispensa, Meal Plan e Spesa.

Ricette, Dispensa e Meal Plan hanno però uno stack interno. Così la navigazione tra le sezioni resta separata da quella interna a ciascuna, senza mescolare i due livelli. I dati passano da una schermata all'altra con i parametri di navigazione: l'id della ricetta, per esempio, viaggia dalla lista al dettaglio e al form di modifica.

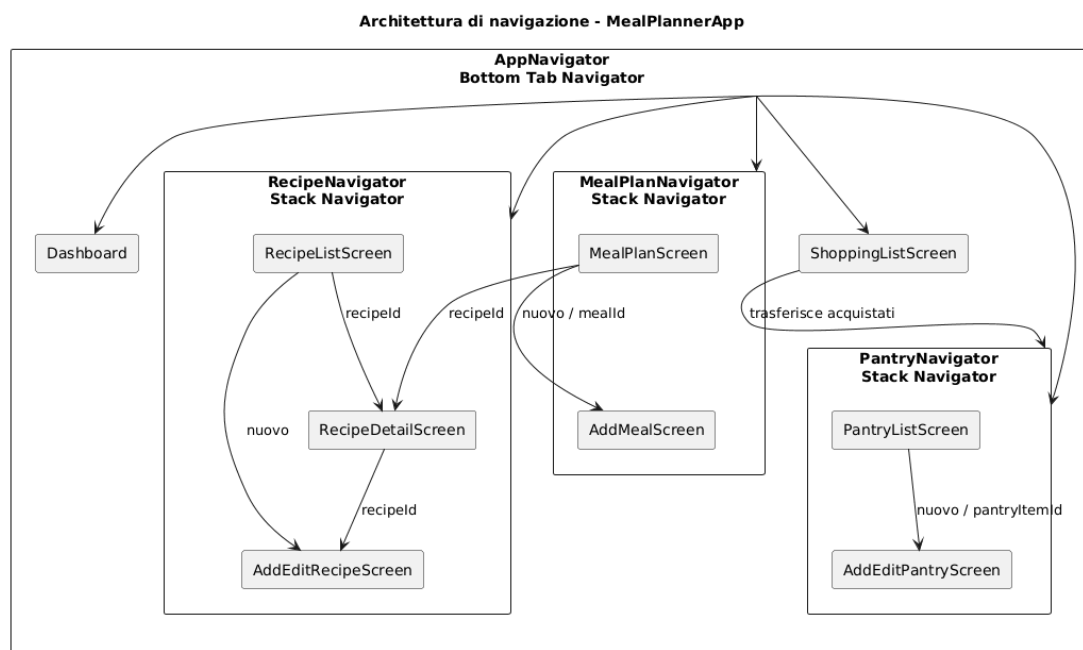


Figura 2: Architettura di navigazione dell'applicazione

3.3 Modello dati

Il modello dati è in TypeScript. Le entità principali sono cinque:

- **Recipe**: una ricetta, con i suoi metadati e la lista degli ingredienti;
- **Ingredient**: nome, quantità e unità richiesti da una ricetta;
- **PantryItem**: un prodotto disponibile in casa;
- **PlannedMeal**: il legame tra una ricetta, una data e un tipo di pasto;
- **ShoppingListItem**: un elemento della lista della spesa, eventualmente generato dal meal plan.

Categorie, difficoltà, unità di misura e tipi di pasto sono union type. Così i valori fuori dominio vengono bloccati già in compilazione e il modello resta esplicito.

Tabella 4: Entità principali del dominio

Entità	Responsabilità
Recipe	Rappresenta ricette e ingredienti necessari.
PantryItem	Rappresenta prodotti disponibili, quantità e scadenze.
PlannedMeal	Associa una ricetta a data e tipo di pasto.
ShoppingListItem	Rappresenta elementi manuali o generati automaticamente.

3.4 Flusso dei dati

Il flusso che lega le aree dell'app è questo:

1. l'utente crea o modifica le ricette;
2. aggiorna la dispensa con i prodotti disponibili;
3. pianifica i pasti scegliendo le ricette;
4. l'app confronta gli ingredienti richiesti con i prodotti in dispensa;
5. la lista della spesa viene generata con i soli ingredienti mancanti;
6. gli articoli comprati vengono spostati in dispensa.

Il vantaggio di questo schema è il riuso. Una ricetta non resta nella sua schermata: alimenta il meal plan, la dashboard e la generazione della lista della spesa.

4 Interfaccia e schermate

Gli screenshot sono presi da dispositivo, in verticale, e messi accanto alla sezione che descrivono. Dove ha senso, la vista principale di una sezione è affiancata alla schermata di inserimento o modifica, così si vedono anche le operazioni CRUD.

4.1 Dashboard

La dashboard è la prima schermata e serve a dare un quadro veloce. Mostra numero di ricette, pasti pianificati nella settimana, prodotti in scadenza, tempo medio di preparazione e categoria più frequente. Sotto compaiono i pasti del giorno, gli ingredienti più usati e i prodotti che mancano per il piano settimanale.

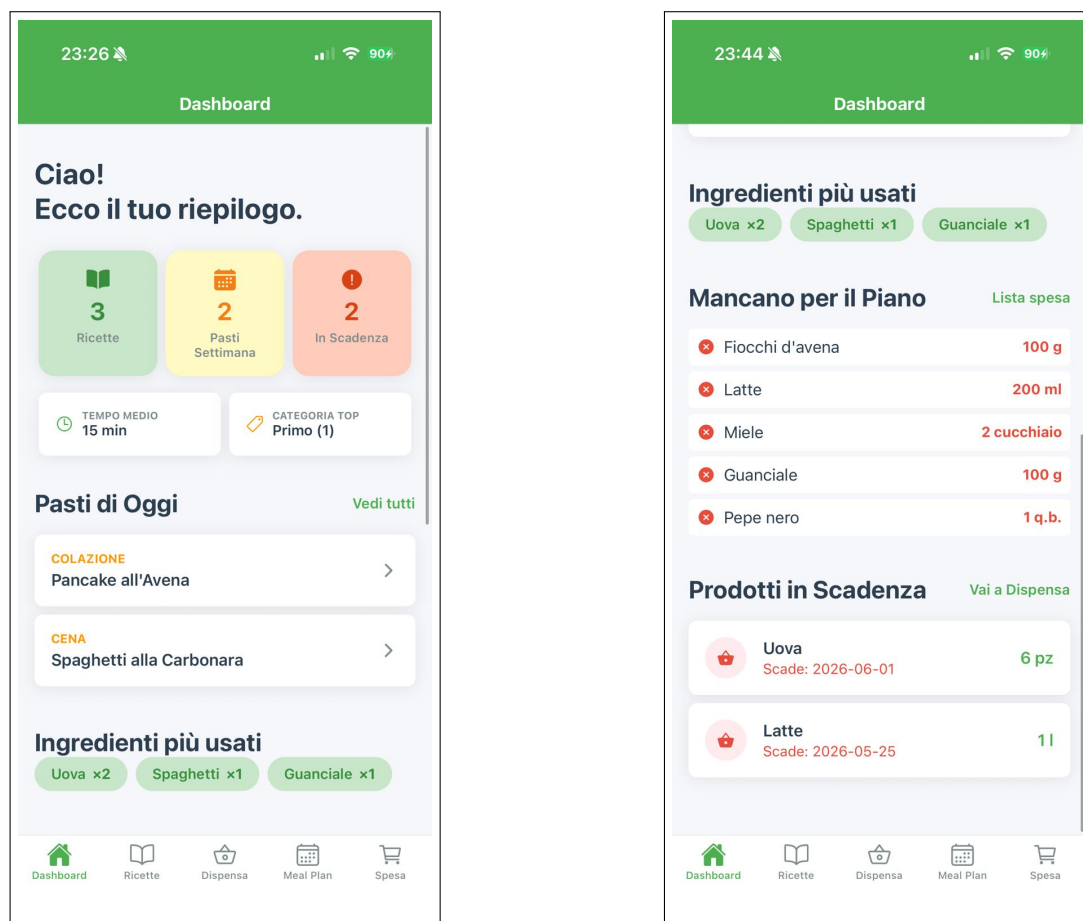


Figura 3: Dashboard: sintesi e riepiloghi

4.2 Gestione delle ricette

La sezione Ricette elenca le ricette salvate. Ogni scheda riporta immagine, nome, categoria, tempo di preparazione e difficoltà; la barra di ricerca filtra per nome o categoria. Il filtro “Cosa posso cucinare oggi?” confronta gli ingredienti di ogni ricetta con quelli in dispensa.

Il dettaglio mostra immagine, categoria, metadati, descrizione, ingredienti e note, e da qui si passa a modifica o eliminazione. Lo stesso form serve per creare e per modificare: in modifica i campi partono già compilati con i dati della ricetta selezionata.

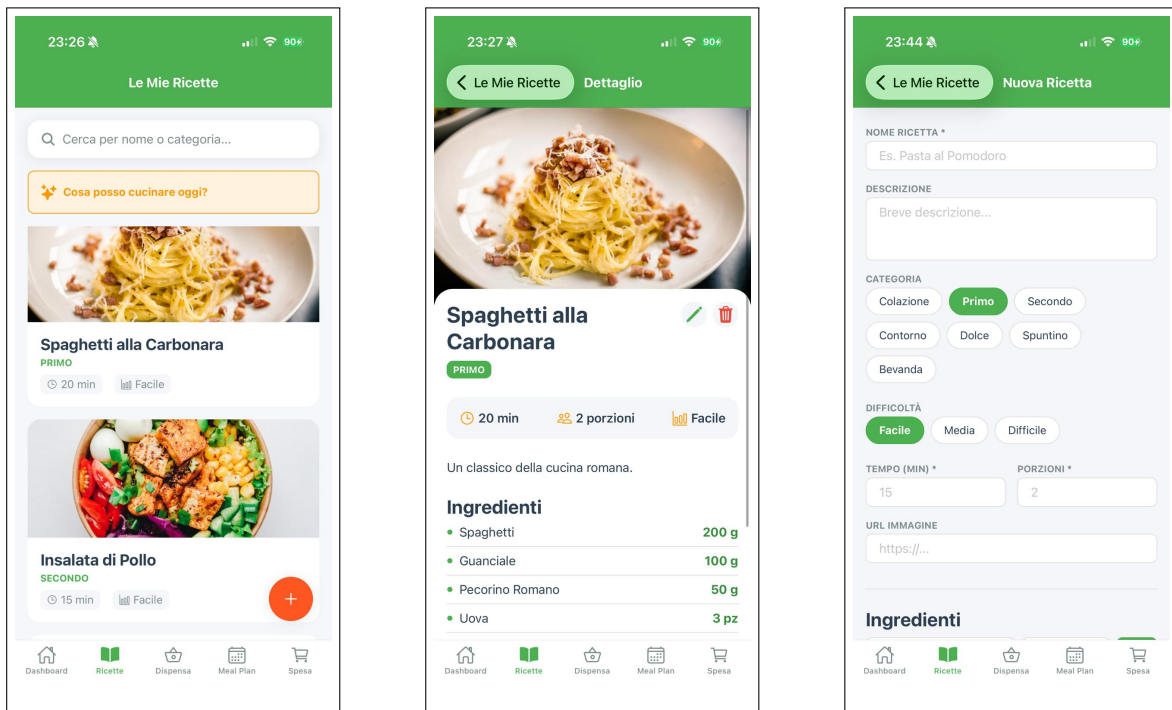


Figura 4: Ricette: elenco, dettaglio e form di inserimento/modifica

4.3 Gestione della dispensa

La sezione Dispensa elenca i prodotti disponibili. La ricerca lavora su nome e categoria; il filtro “Solo prodotti in scadenza” isola quelli già scaduti o in scadenza entro sette giorni. Ogni voce mostra quantità, unità di misura e stato della scadenza.

Nel form si inseriscono nome, categoria, quantità, unità di misura, data di scadenza e note. Se la data manca, lo store ne stima una in base al tipo di prodotto: più vicina per i freschi, più lontana per i secchi e i prodotti a lunga conservazione.

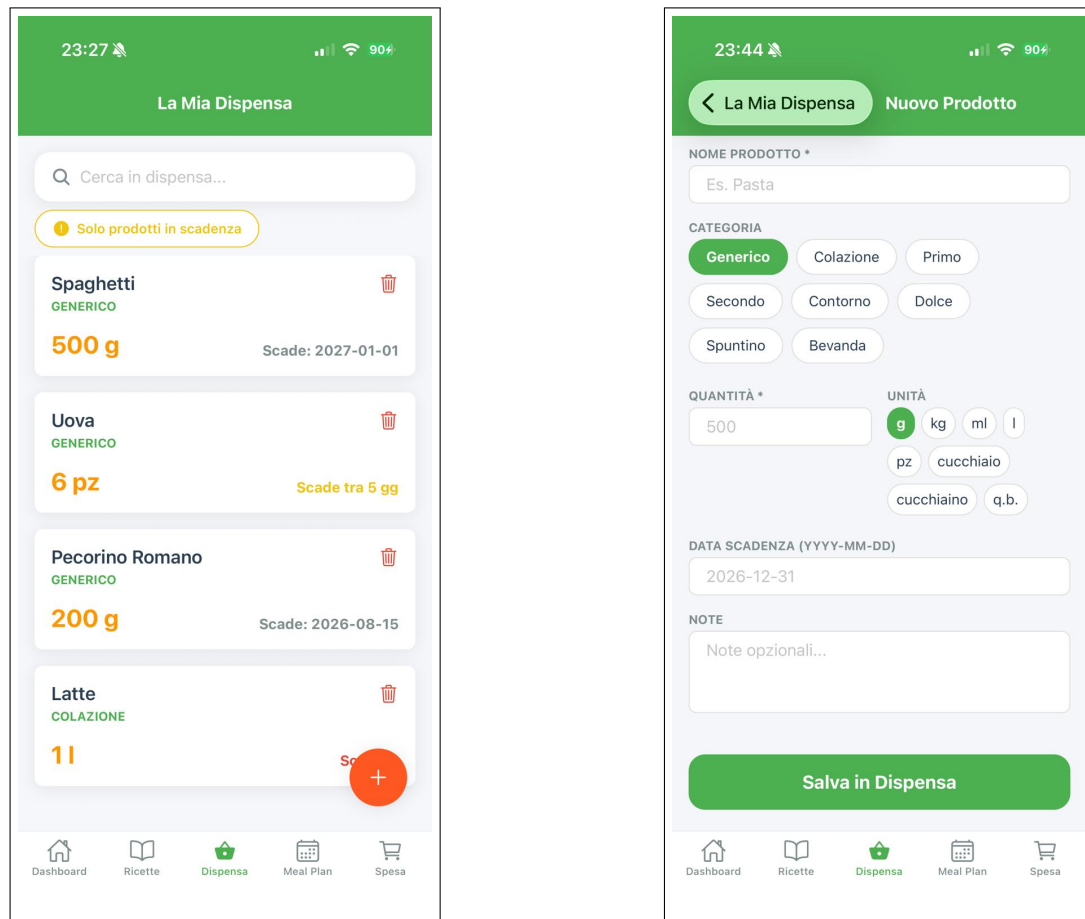


Figura 5: Dispensa: consultazione e operazioni

4.4 Pianificazione dei pasti

La sezione Meal Plan parte dalla data corrente e mostra i sette giorni successivi. Per ogni giorno ci sono i pasti pianificati, divisi tra colazione, spuntino, pranzo e cena. Ogni pasto tiene il riferimento alla ricetta e si può modificare o eliminare.

Il form chiede data, tipo di pasto e ricetta. Fermare la vista principale a sette giorni tiene l'interfaccia leggibile e in linea con una pianificazione settimanale.

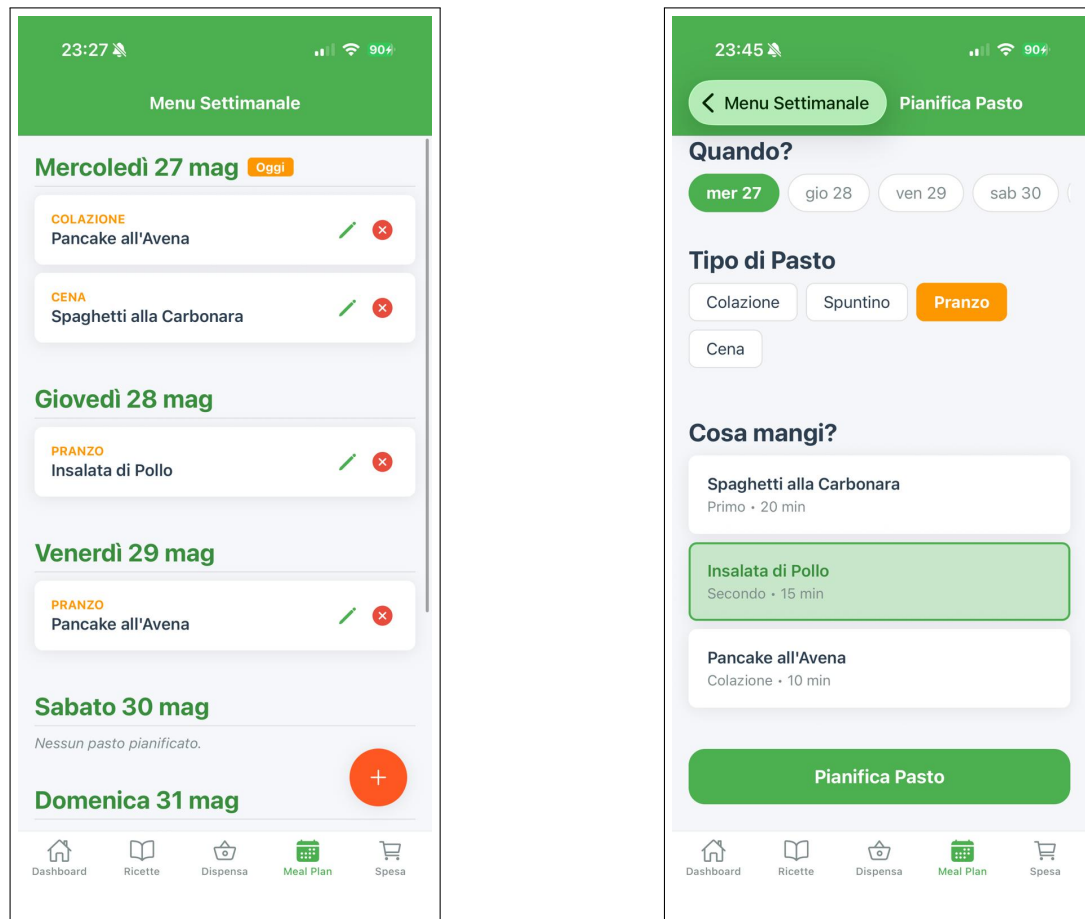


Figura 6: Meal plan: pianificazione e modifica

4.5 Lista della spesa

La lista della spesa permette inserimento manuale, modifica in linea, eliminazione e marcatura degli acquistati. Le voci create in automatico hanno un badge “Auto” che le distingue da quelle inserite a mano.

“Genera da Meal Plan” raccoglie gli ingredienti delle ricette pianificate. “Sposta acquistati in Dispensa” aggiorna la dispensa: se un prodotto c’è già con lo stesso nome e la stessa unità di misura, le quantità si sommano; altrimenti nasce un nuovo prodotto generico.

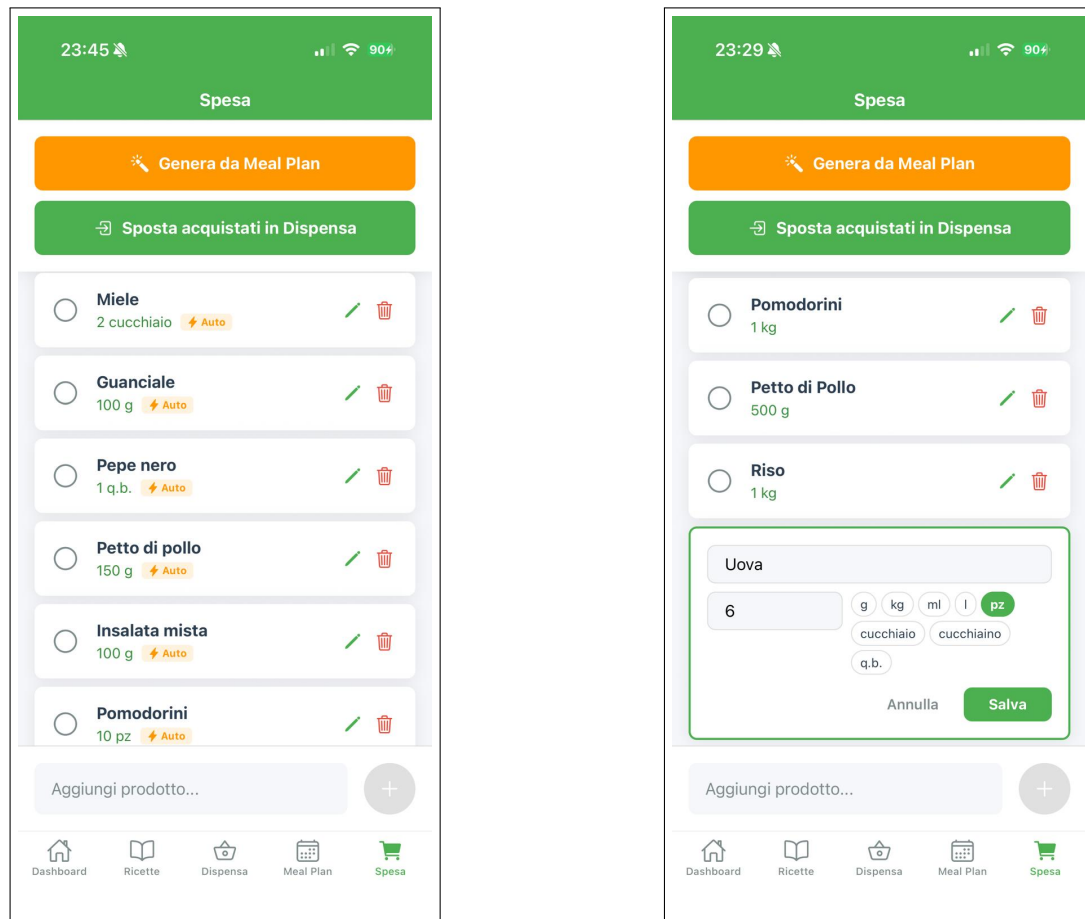


Figura 7: Lista della spesa: gestione e modifica

5 Scelte tecnologiche

5.1 Framework e versione Expo

Il progetto gira su Expo SDK 54, React Native 0.81.5 e React 19.1.0. La documentazione di Expo SDK 54 conferma l'abbinamento con React Native 0.81 e React 19.1.0, cioè quello dichiarato in `package.json`. Pagina di riferimento: <https://docs.expo.dev/versions/v54.0.0/>.

Expo lo abbiamo scelto per un motivo pratico: riduce la configurazione nativa iniziale e fa partire e mostrare l'app in fretta, lasciando spazio alla logica applicativa. In `app.json` sono attivi orientamento portrait, interfaccia light, nuova architettura React Native, supporto tablet iOS ed edge-to-edge su Android.

5.2 Librerie principali

Libreria	Uso nel progetto	Motivo
React Navigation	Navigazione tab e stack tra le schermate.	Standard di fatto in React Native
Zustand	Store per ricette, dispensa, meal plan e spesa.	Stato leggero, poco boiler-plate

Libreria	Uso nel progetto	Motivo
AsyncStorage	Persistenza locale tramite middleware Zustand.	Salvataggio su dispositivo
Expo Vector Icons	Icone Ionicons in tab, bottoni, card e stati.	Interfaccia più chiara
UUID e random values	Identificativi univoci per i nuovi elementi.	Riferimenti stabili ai dati
TypeScript	Tipizzazione del dominio e delle strutture dati.	Più manutenibilità

5.3 Gestione della persistenza

La persistenza passa dal middleware **persist** di Zustand, con serializzazione JSON su **AsyncStorage**. Ogni dominio ha la sua chiave:

- **recipe-storage** per le ricette;
- **pantry-storage** per la dispensa;
- **mealplan-storage** per la pianificazione;
- **shoppinglist-storage** per la lista della spesa.

Tenere le chiavi separate evita che i dati dei diversi domini si sovrappongano e rende più semplice intervenire su uno solo. Gli store partono con dati mock, utili a mostrare le funzioni durante la presentazione.

6 Implementazione

6.1 Organizzazione del codice

Il codice è diviso per responsabilità. Le schermate gestiscono interfaccia, form e interazioni; gli store concentrano CRUD e persistenza; i tipi TypeScript descrivono le entità; il tema raccoglie colori, spaziature, bordi, tipografia e ombre.

Non c'è un layer repository separato: i dati sono pochi e la persistenza è locale, quindi non serviva. Con un backend o una sincronizzazione cloud il quadro sarebbe diverso, e converrebbe separare meglio accesso ai dati, cache e logica di sincronizzazione.

6.2 Gestione dello stato

Ogni store Zustand espone stato e funzioni di modifica del proprio dominio:

- **useRecipeStore**: aggiunta, modifica e cancellazione delle ricette;
- **usePantryStore**: aggiunta, modifica e cancellazione dei prodotti in dispensa;
- **useMealPlanStore**: aggiunta, modifica e cancellazione dei pasti pianificati;
- **useShoppingListStore**: aggiunta, modifica, eliminazione, marcatura come acquistato e sostituzione della lista.

Le nuove entità prendono un **id** da **uuidv4**. Gli aggiornamenti sono immutabili: gli array si ricreano con **map**, **filter** o **spread**, come di prassi in React.

6.3 CRUD ricette

La lista ricette usa `FlatList` per le card e `TextInput` per la ricerca. Il dettaglio recupera la ricetta dall'id passato nei parametri di navigazione. Il form `AddEditRecipeScreen` è lo stesso per creazione e modifica: se arriva un `id`, i campi partono dai dati esistenti e il salvataggio chiama `updateRecipe`; altrimenti chiama `addRecipe`.

Gli ingredienti vivono in uno stato locale del form finché non si salva. In questo modo l'utente li aggiunge o li toglie mentre compila, senza toccare lo store prima della conferma.

6.4 CRUD dispensa

La dispensa segue lo stesso schema. La lista offre ricerca, filtro e cancellazione diretta; il form `AddEditPantryScreen` gestisce nome, categoria, quantità, unità di misura, scadenza e note. Lo store contiene anche la stima della scadenza per quando l'utente non la inserisce: data più vicina per i freschi, più lontana per i secchi.

Lo stesso controllo sulle scadenze serve sia alla lista sia alla dashboard: i prodotti scaduti o in scadenza entro sette giorni vengono segnalati.

6.5 Pianificazione dei pasti

Il meal plan calcola i sette giorni al volo, invece di salvare una struttura a calendario che sarebbe ridondante. In memoria restano solo i pasti pianificati, ognuno con data, tipo e ricetta; in visualizzazione, per ogni giorno si filtrano i pasti che gli appartengono.

Selezionando un pasto si può aprire il dettaglio della ricetta saltando allo stack Ricette. È un caso che mostra bene come i dati passino tra schermate e come le sezioni si integrino tra loro.

6.6 Lista della spesa automatica

La generazione automatica della lista della spesa è la componente più articolata dell'implementazione. L'algoritmo:

1. itera sui pasti pianificati;
2. recupera la ricetta associata;
3. somma gli ingredienti richiesti usando una chiave fatta di nome e unità;
4. sottrae le quantità disponibili in dispensa;
5. scarta gli ingredienti già coperti dalla dispensa;
6. tiene gli elementi manuali non ancora acquistati;
7. sostituisce gli elementi automatici con quelli appena calcolati.

Usare la coppia nome/unità evita confronti ambigui, ma porta con sé un limite dichiarato: unità diverse non vengono convertite. Per il progetto va bene così, perché la traccia non chiede una gestione completa delle conversioni reali.

6.7 Dashboard e statistiche

La dashboard calcola i valori derivati con `useMemo`:

- pasti del giorno;
- numero di pasti nella settimana corrente;
- prodotti in scadenza;

- tempo medio di preparazione delle ricette;
- categoria di ricette più frequente;
- ingredienti più usati;
- prodotti mancanti per i pasti pianificati.

Con questi numeri la schermata iniziale dà subito il quadro: l'utente non deve aprire una sezione alla volta per capire come stanno le cose.

7 Conclusioni

MealPlannerApp copre i requisiti della traccia e li tiene insieme in un flusso unico. Il cuore del progetto è proprio l'interazione tra i dati: ricette, dispensa, meal plan e lista della spesa non sono moduli a sè, ma pezzi dello stesso ciclo.

Le scelte tecnologiche reggono bene il contesto mobile. Expo semplifica esecuzione e presentazione; React Navigation tiene una navigazione multi-schermata chiara; Zustand dà uno stato leggero e persistente; AsyncStorage salva in locale senza backend. TypeScript rende esplicite le entità e taglia buona parte degli errori nella manipolazione dei dati.

I limiti noti — niente conversione tra unità di misura, date inserite a mano — sono coerenti con il taglio didattico. Da qui si potrebbe crescere con notifiche, sincronizzazione cloud, importazione dei prodotti, un calendario più esteso e una gestione delle quantità più completa.